

unb-ppgi0119

Disciplina de Tópicos em Fundamentos e Métodos da Computação

Atividade 03 (Criação de Modelo de Logistic Regression)

Ao rodar o script `find_best_hyperparameters.py` é gerado um `performance_log.csv`

Step	Elapsed Time (s)	CPU Usage (%)	Memory Usage (%)
Data Loading	0.28	13.7	22.4
Class Filtering	0.29	0.0	22.4
Data Splitting	1.22	29.5	22.4
Vectorization	3.97	14.7	22.8
Greedy Search - MultinomialNB	5.83	34.5	25.0
Greedy Search - LogisticRegression	638.28	80.1	25.1
Results Saving	638.29	0.0	25.1
Script Completion	638.29	0.0	25.1

https://github.com/thiagodepaulo/nlp/blob/main/aula_2/exercicio2.md

Já estão disponíveis as duas bases de dados para os experimentos do Exercício 3. Por favor, façam novos notebooks (pode ser um para cada base ou um para todas) com os resultados de classificação.

Link da planilha:

<https://docs.google.com/spreadsheets/d/1F485czBA5zR60J4efsEz8-4YydywazOPCKVjb1o0jUs/edit?gid=832486682#gid=832486682>

PROF

Orientação de Exercícios de PLN

Objetivo

Os alunos devem implementar e treinar classificadores de texto utilizando alguma das seguintes bases: [Text Collections](#).

Duas bases de dados serão escolhidas e descritas na planilha [Aqui](#). As bases escolhidas para cada aluno serão disponibilizadas um dia antes da entrega. Façam seus códigos e experimentos de forma que seja rápido e fácil executar para alguma das bases escolhidas no dia anterior.

Data da entrega: **26/11/2024**

Ferramentas e Bibliotecas

- **Linguagem:** Python
- **Bibliotecas principais:**
 - **sklearn** para criação e treinamento dos modelos
 - **pandas** para manipulação de dados

Classificadores e Abordagem

1. Modelos a implementar:

- Multinomial Naive Bayes
- Logistic Regression
- Um outro modelo da sua escolha

2. Otimização de hiperparâmetros:

- Aplicar **Greedy Search** na base de treino para encontrar os melhores parâmetros.
- Os alunos devem estudar o significado dos parâmetros relevantes e pesquisar um conjunto adequado para o Greedy Search.

3. Divisão da base:

- Divida a base em treino (80%) e teste (20%).
- Realize a busca dos melhores hiperparâmetros na base de treino.

4. Validação:

- Após encontrar os melhores parâmetros, re-treine o modelo com todos os dados de treino.
- Calcule as métricas **F1 Score** (macro e micro) e **Acurácia** no conjunto de teste.

Passos Detalhados

Parte 1: Implementação do Script para Greedy Search

1. Crie um arquivo **find_best_hyperparameters.py** que:

- Carregue a base de dados especificada.
- Divida a base em treino e teste (80/20).
- Salve a base de treino e teste para a Avaliação Final.
- Realize Greedy Search nos hiperparâmetros relevantes para cada modelo.
- Salve todos os resultados dos parâmetros avaliados em um **DataFrame** do pandas.
- Exporte o DataFrame com os resultados para um arquivo **.csv**.

Parte 2: Notebook para Avaliação Final

1. Crie um **notebook** que:

- Carregue a base de dados e os resultados do Greedy Search exportados pelo script.
- Apresente o DataFrame com os resultados do Greedy Search.
- Treine os modelos com os melhores hiperparâmetros encontrados na base de treino.
- Calcule e exiba as métricas **F1 Score** (macro e micro) e **Acurácia** no conjunto de teste.
- Interpretem os resultados e justifiquem os melhores hiperparâmetros encontrados!

Entregáveis

1. Arquivo Python:

- `find_best_hyperparameters.py` para realizar Greedy Search.
- Inclua comentários explicativos sobre o código e os parâmetros usados.

2. Notebook:

- Apresente:
 - O DataFrame com os resultados da busca de hiperparâmetros.
 - As métricas finais calculadas no conjunto de teste.

3. Arquivo CSV:

- Exportação do DataFrame com os resultados do Greedy Search.
-